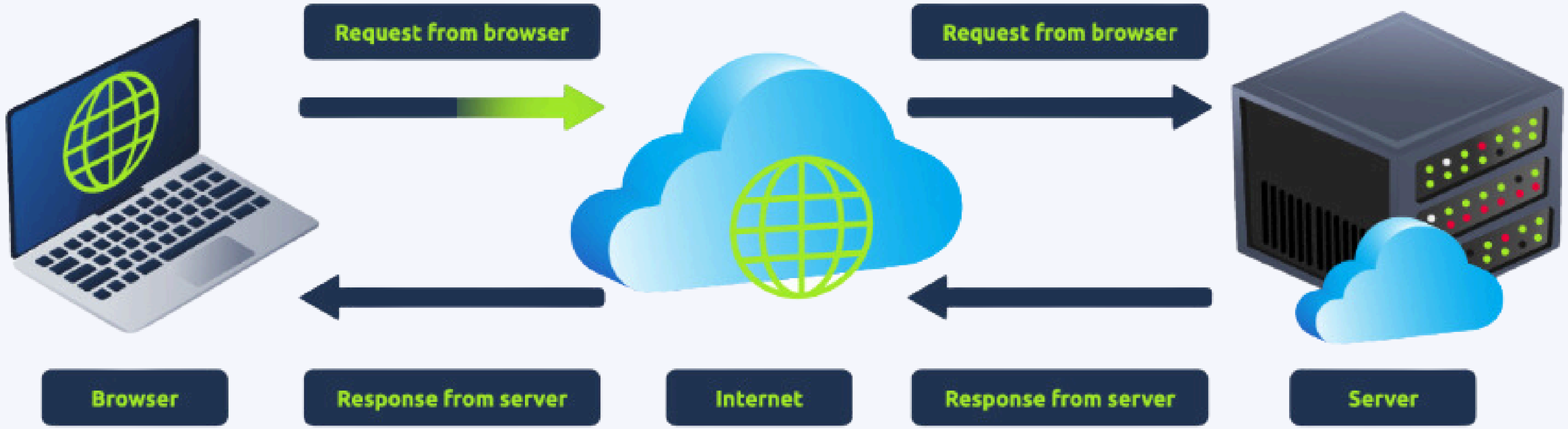
Web Application Fundamentals & Pentesting

PRESENTATION BY HARSHIT BHARDWAJ

Content

- How Website Works?
- DNS (Domain Name System)

How Website works?



How Website works?

When you visit a website, your browser (like Safari or Google Chrome) makes a request to a web server asking for information about the page you're visiting. It will respond with data that your browser uses to show you the page; a web server is just a dedicated computer somewhere else in the world that handles your requests.

How Website works?

There are two major components that make up a website:

- Front End (Client-Side) - the way your browser renders a website.
- Back End (Server-Side) - a server that processes your request and returns a response.

There are many other processes involved in your browser making a request to a web server, but for now, you just need to understand that you make a request to a server, and it responds with data your browser uses to render information to you.

How Website works?

Websites are primarily created using:

- **HTML, to build websites and define their structure**
- **CSS, to make websites look pretty by adding styling options**
- **JavaScript, implement complex features on pages using interactivity**

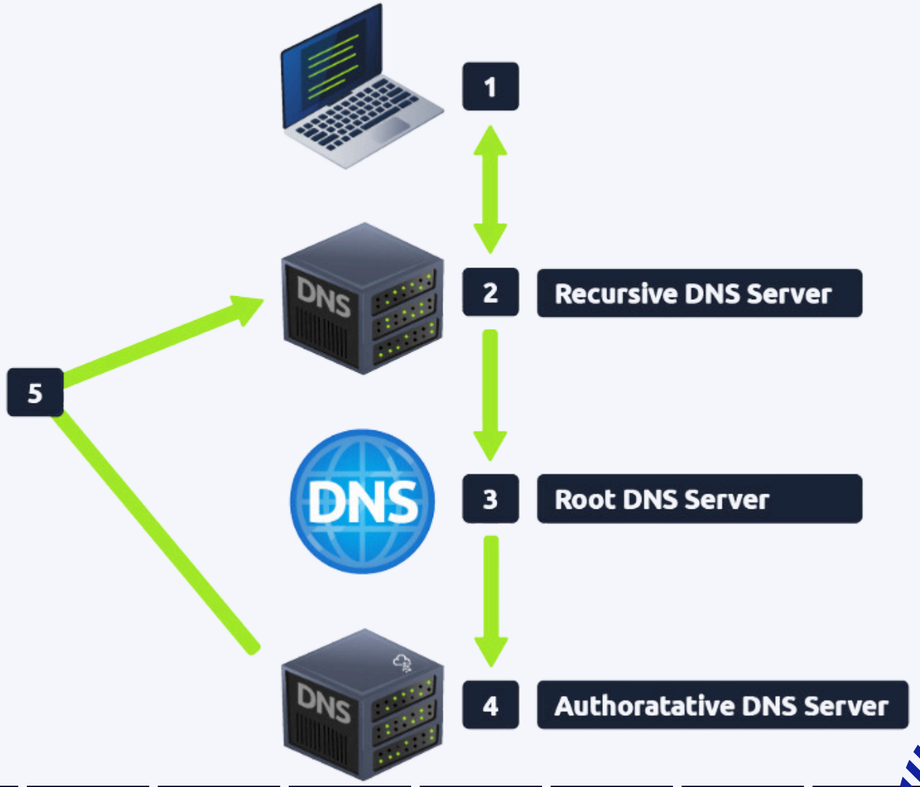
DNS

DNS, or the Domain Name System, translates human readable domain names (for example, www.amazon.com) to machine readable IP addresses (for example, 192.0.2.44).

Much like every house has a unique address for sending mail directly to it, every computer on the internet has its own unique address to communicate with it called an IP address. An IP address looks like the following 104.26.10.229, 4 sets of digits ranging from 0 - 255 separated by a period.

Making a DNS Request.

1. When you request a domain name, your computer first checks its local cache to see if you've previously looked up the address recently; if not, a request to your Recursive DNS Server will be made.



Making a DNS Request.

2. A Recursive DNS Server is usually provided by your ISP, but you can also choose your own. This server also has a local cache of recently looked up domain names. If a result is found locally, this is sent back to your computer, and your request ends here. If the request cannot be found locally, a journey begins to find the correct answer, starting with the internet's root DNS servers.

3. The root servers act as the DNS backbone of the internet; their job is to redirect you to the correct Top Level Domain Server, depending on your request. If, for example, you request www.tryhackme.com, the root server will recognise the Top Level Domain of .com and refer you to the correct TLD server that deals with .com addresses.

Making a DNS Request.

2. A Recursive DNS Server is usually provided by your ISP, but you can also choose your own. This server also has a local cache of recently looked up domain names. If a result is found locally, this is sent back to your computer, and your request ends here. If the request cannot be found locally, a journey begins to find the correct answer, starting with the internet's root DNS servers.

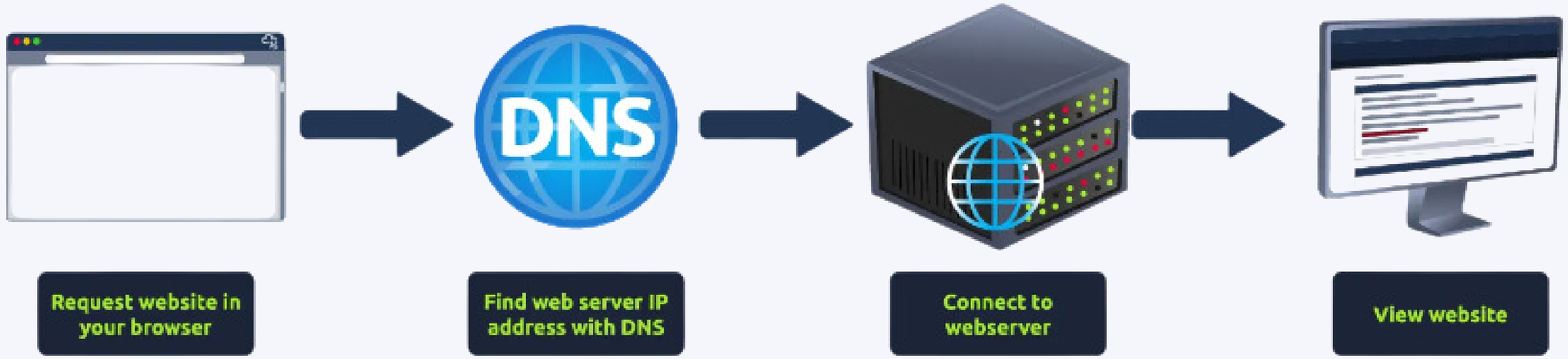
3. The root servers act as the DNS backbone of the internet; their job is to redirect you to the correct Top Level Domain Server, depending on your request. If, for example, you request www.tryhackme.com, the root server will recognise the Top Level Domain of .com and refer you to the correct TLD server that deals with .com addresses.

Making a DNS Request.

4. The TLD server holds records for where to find the authoritative server to answer the DNS request. The authoritative server is often also known as the nameserver for the domain. For example, the name server for tryhackme.com is kip.ns.cloudflare.com and uma.ns.cloudflare.com. You'll often find multiple nameservers for a domain name to act as a backup in case one goes down.

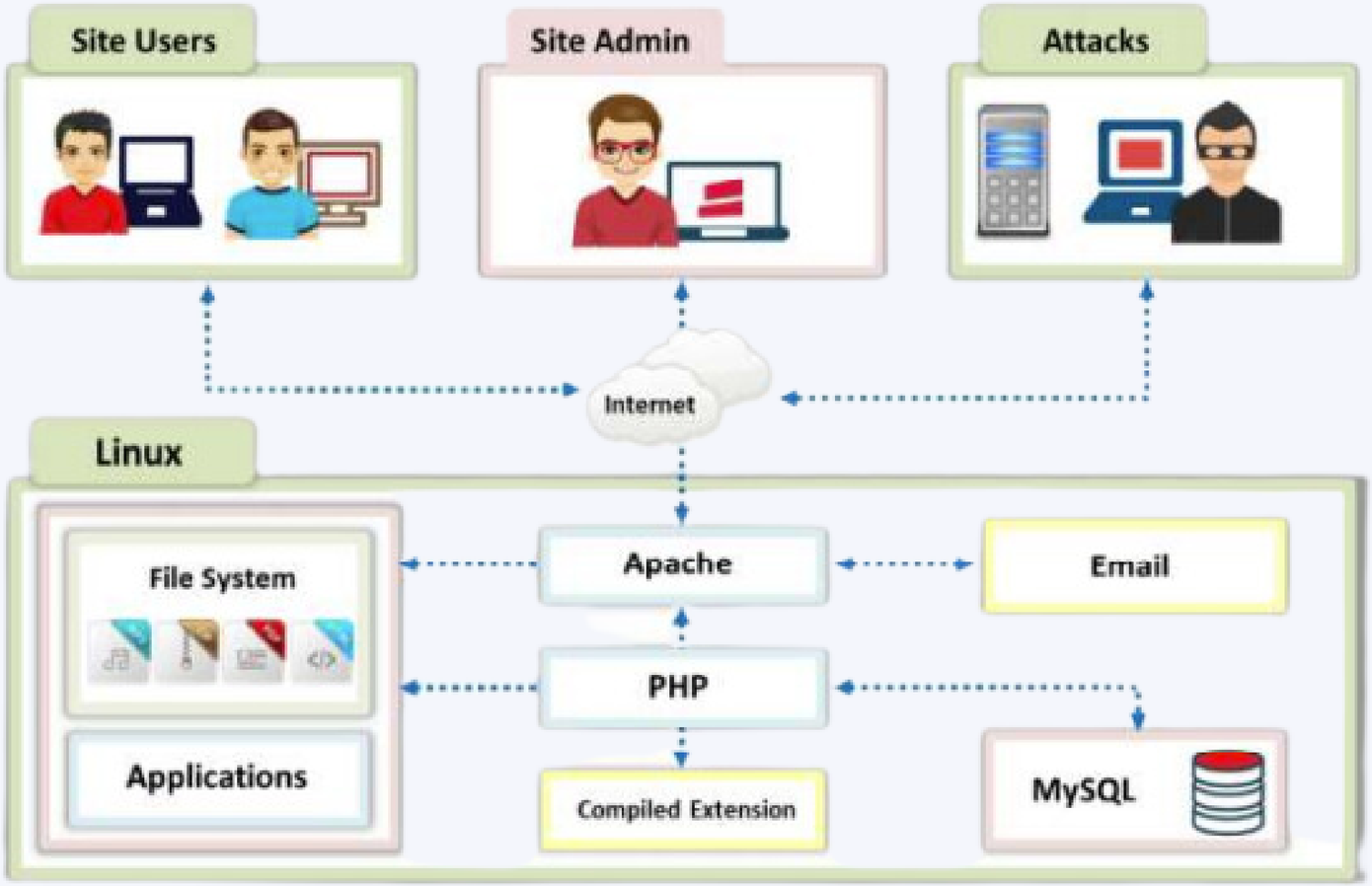
5. An authoritative DNS server is the server that is responsible for storing the DNS records for a particular domain name and where any updates to your domain name DNS records would be made.

How Website works?



Presentation by Harshit Bhardwaj

Open Source Web Server Architecture



Presentation by Harshit Bhardwaj

Presentation by Harshit Bhardwaj

Open Source Web Server Architecture

Open-source Web Server Architecture

Open-source web server architecture typically uses Linux, Apache, MySQL, and PHP, often called the LAMP software bundle, as the principal components.

The following are the functions of the principal components in open-source web server architecture:

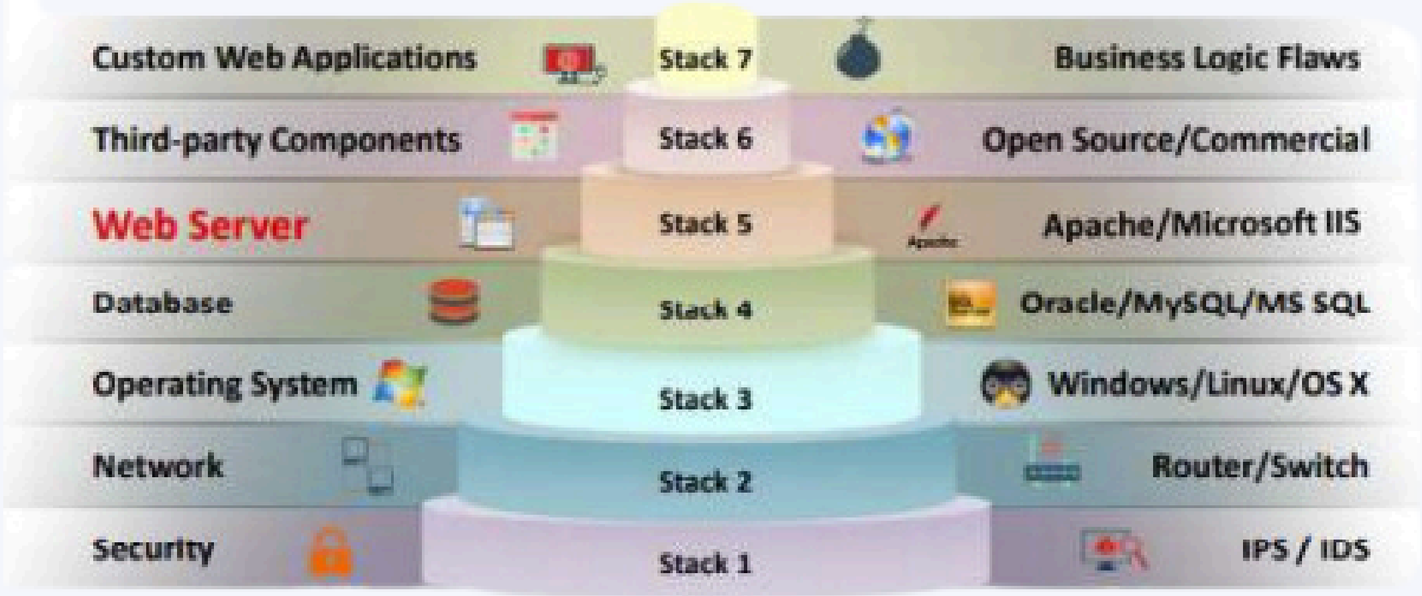
- Linux is the operating system (OS) of the web server and provides a secure platform
- Apache is the component of the web server that handles each HTTP request and response
- MySQL is a relational database used to store the content and configuration information of the web server
- PHP is the application layer technology used to generate dynamic web content

Presentation by Harshit Bhardwaj

Presentation by Harshit Bhardwaj

Web Server Security Issues

- Attackers usually target **software vulnerabilities** and configuration errors to compromise web servers
- **Network** and **OS level attacks** can be well defended using proper network security measures such as firewalls, IDS, etc. However, web servers can be accessed from anywhere via the Internet, which renders them **highly vulnerable** to attacks



Impact of Web Server Attacks

- Compromise of user accounts
- Website defacement
- Secondary attacks from the website
- Root access to other applications or servers
- Data tampering and data theft
- Reputational damage of the company

Presentation by Harshit Bhardwaj

Presentation by Harshit Bhardwaj

Web Server Security Issues

Organizations can defend most network-level and OS-level attacks by adopting network security measures such as firewalls, intrusion detection systems (IDSs), and intrusion prevention systems (IPSs) and by following security standards and guidelines. This forces attackers to turn their attention to web-server- and web-application-level attacks because a web server that hosts web applications is accessible from anywhere over the Internet. This makes web servers an attractive target. Poorly configured web servers can create vulnerabilities in even the most carefully designed firewall systems. Attackers can exploit poorly configured web servers with known vulnerabilities to compromise the security of web applications. Furthermore, web servers with known vulnerabilities can harm the security of an organization. As shown in below figure, organizational security includes seven levels from stack 1 to stack 7.

Presentation by Harshit Bhardwaj

Presentation by Harshit Bhardwaj

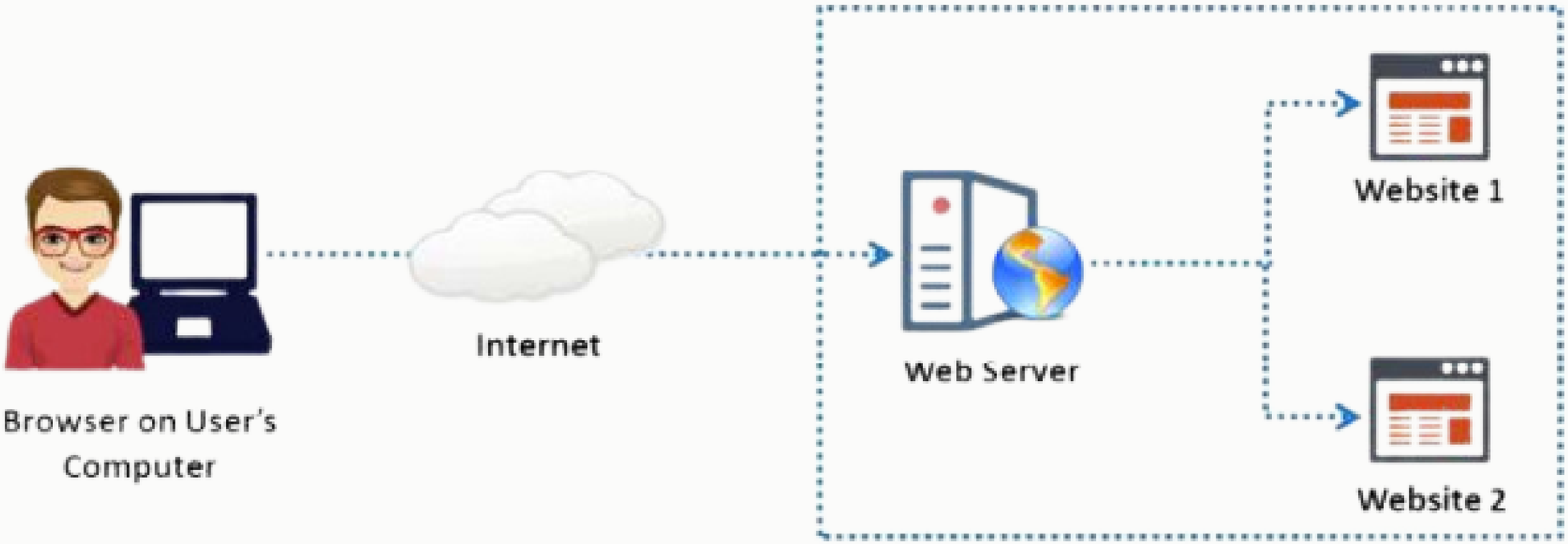
Web Server Security Issues

A web server is a hardware/software application that hosts websites and makes them accessible over the Internet. A web server, along with a browser, successfully implements client–server model architecture. In this model, the web server plays the role of the server, and the browser acts as the client. To host websites, a web server stores the web pages of websites and delivers a particular web page upon request. Each web server has a domain name and an IP address associated with that domain name. A web server can host more than one website. Any computer can act as a web server if it has specific server software (a web server program) installed and is connected to the Internet.

Web servers are chosen based on their capability to handle server-side programming, security characteristics, publishing, search engines, and site-building tools. Apache, Microsoft IIS, Nginx, Google, and Tomcat are some of the most widely used web server software. An attacker usually targets vulnerabilities in the software component and configuration errors to compromise web servers.

Presentation by Harshit Bhardwaj

Web Server Security Issues



Conceptual diagram of a web server: the user visits websites hosted on a web server

Presentation by Harshit Bhardwaj

Common Goals behind Web Server Hacking

Attackers perform web server attacks with certain goals in mind. These goals may be either technical or non-technical. For example, attackers may breach the security of a web server and steal sensitive information for financial gains or merely for the sake of curiosity.

The following are some common goals of web server attacks:

- Stealing credit-card details or other sensitive credentials using phishing techniques
- Integrating the server into a botnet to perform denial of service (DoS) or distributed DoS (DDoS) attacks
- Compromising a database
- Obtaining closed-source applications
- Hiding and redirecting traffic
- Escalating privileges

Presentation by Harshit Bhardwaj

Presentation by Harshit Bhardwaj

Dangerous Security Flaws Affecting Web Server Security

A web server configured by poorly trained system administrators may have security vulnerabilities. Inadequate knowledge, negligence, laziness, and inattentiveness toward security can pose the greatest threats to web server security.

The following are some common oversights that make a web server vulnerable to attacks:

- Failing to update the web server with the latest patches
- Using the same system administrator credentials everywhere
- Allowing unrestricted internal and outbound traffic
- Running unhardened applications and servers

Presentation by Harshit Bhardwaj

Presentation by Harshit Bhardwaj

Why are Web Servers Compromised?

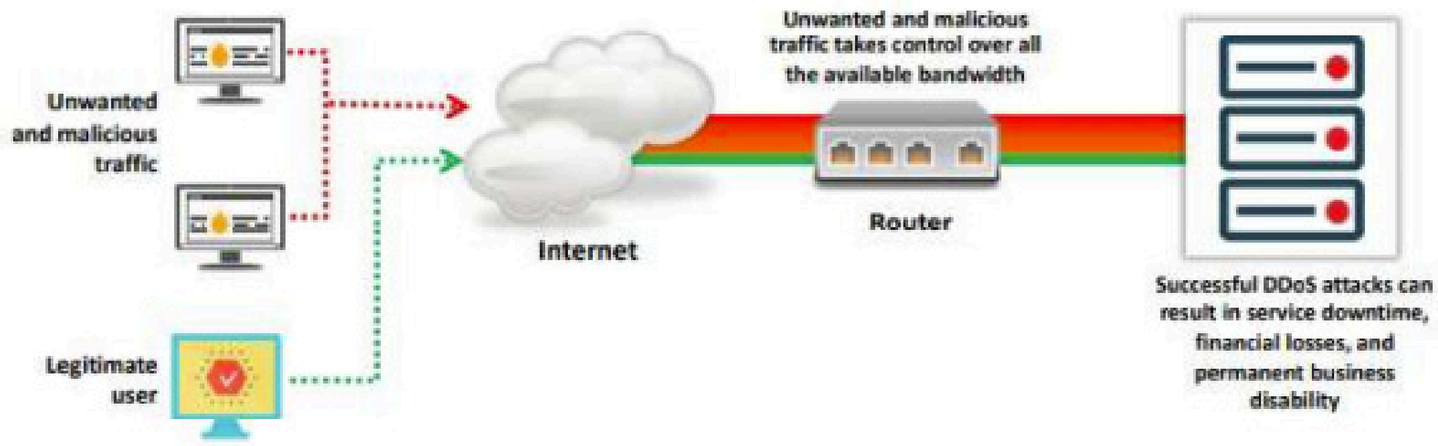
- **Improper** file and directory **permissions**
- Server installation with **default settings**
- Enabling of **unnecessary services**, including content management and remote administration
- **Security conflicts** with business ease-of-use case
- **Lack of proper security policies**, procedures, and maintenance
- **Improper authentication** with external systems
- **Default accounts** having default passwords, or no passwords
- **Unnecessary** default, backup, or sample **files**
- **Misconfigurations** in web server, operating systems, and networks
- **Bugs** in server software, OS, and web applications
- **Misconfigured SSL certificates** and encryption settings
- Administrative or **debugging functions** that are **enabled** or accessible on web servers
- Use of **self-signed certificates** and default certificates

Presentation by Harshit Bhardwaj

Web Server Attacks

DoS/DDoS Attacks

- Attackers may send numerous **fake requests** to the web server, which causes **web server crashing** or makes it unavailable to the legitimate users
- Attackers may target **high profile web servers** such as banks, credit card payment gateways, and government owned services to **steal user credentials**

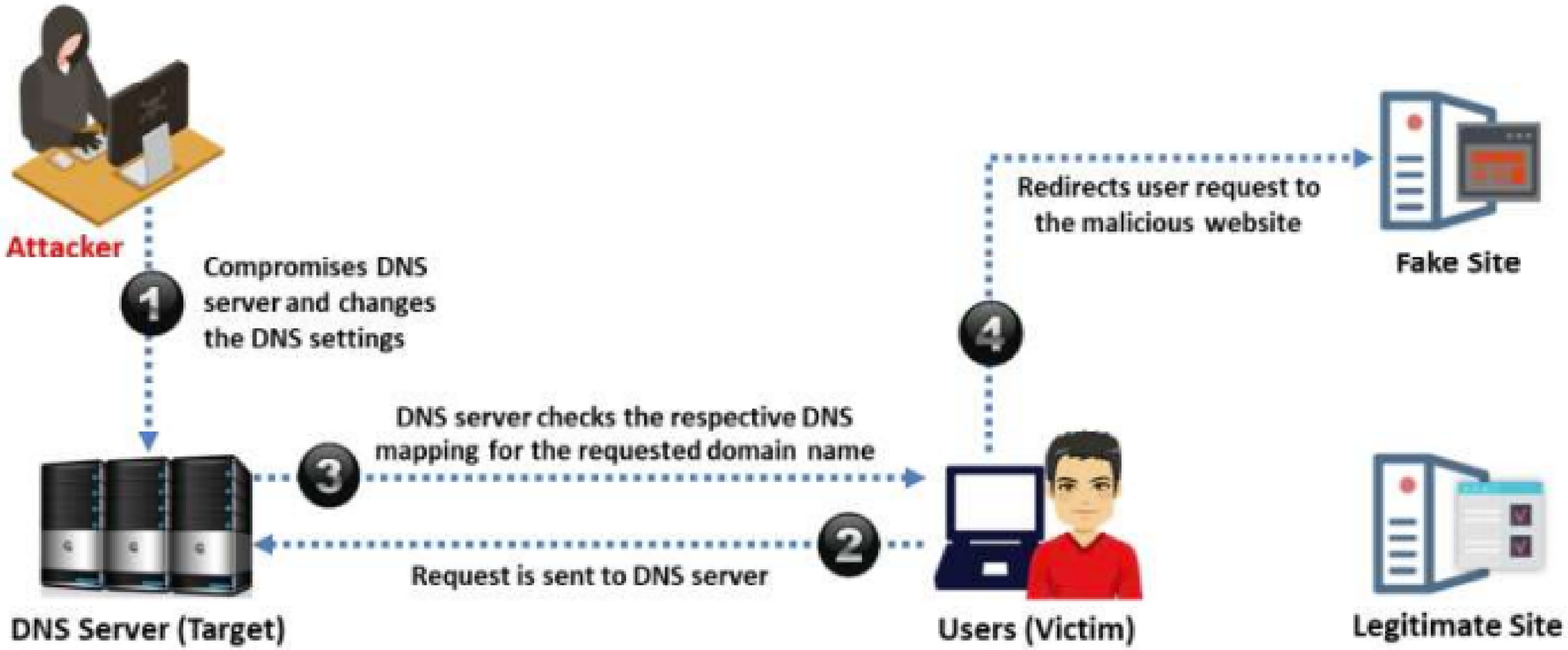


Web Server Attacks

To crash a web server running an application, the attacker targets the following services to consume the web server's resources with fake requests:

- Network bandwidth
- Server memory
- Application exception handling mechanism
- CPU usage
- Hard-disk space
- Database space

Web Server Attacks



Web Server Attacks

The Domain Name System (DNS) resolves a domain name to its corresponding IP address. A user queries the DNS server with a domain name, and the DNS server responds with the corresponding IP address.

In DNS server hijacking, an attacker compromises a DNS server and changes its mapping settings to redirect toward a rogue DNS server that would redirect the user's requests to the attacker's rogue server. Consequently, when the user enters a legitimate URL in a browser, the settings will redirect to the attacker's fake site.

Web Server Attacks

Directory Traversal Attacks

- In directory traversal attacks, attackers use the **../ (dot-dot-slash)** sequence to access restricted directories outside the web server root directory
- Attackers can use the **trial and error method** to navigate outside the root directory and access sensitive information in the system



Web Server Attacks

Directory Traversal Attacks

An attacker may be able to perform a directory traversal attack owing to a vulnerability in the code of a web application. In addition, poorly patched or configured web server software can make the web server vulnerable to a directory traversal attack.

The design of web servers limits public access to some extent. Directory traversal is the exploitation of HTTP through which attackers can access restricted directories and execute commands outside the web server's root directory by manipulating a Uniform Resource Locator (URL). In directory traversal attacks, attackers use the dot-dot-slash (../) sequence to access restricted directories outside the web server's root directory. Attackers can use the trial-and-error method to navigate outside the root directory and access sensitive information in the system.

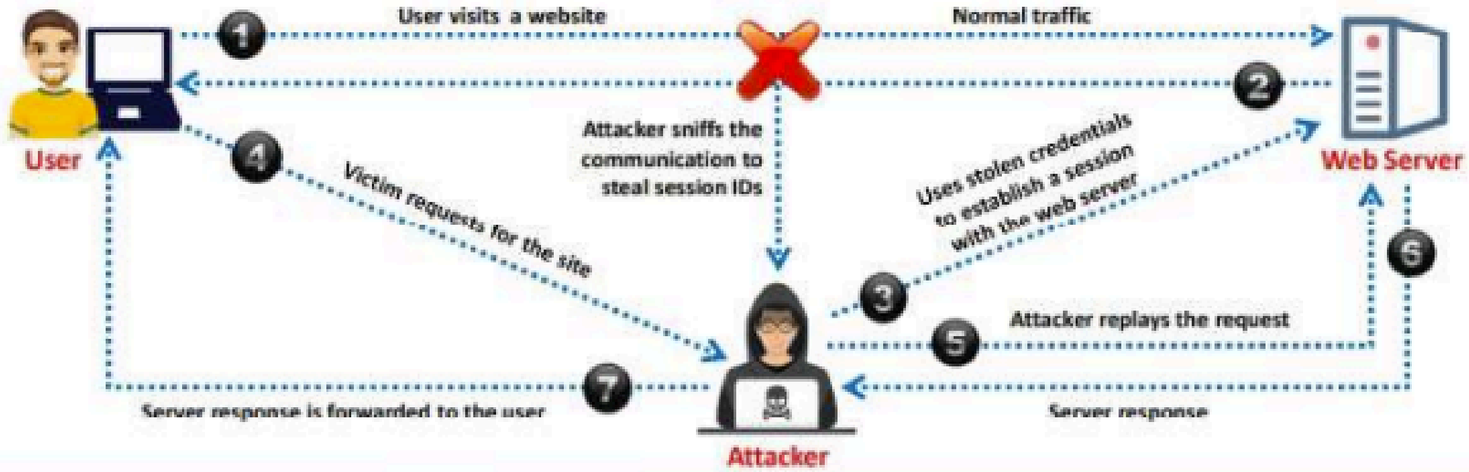
An attacker exploits the web server software (web server program) to perform directory traversal attacks. The attacker usually performs this attack with the help of a browser. A web server is vulnerable to this attack if it accepts input data from a browser without proper validation.

Web Server Attacks

Man-in-the-Middle/Sniffing Attack

1. Man-in-the-Middle (MITM) attacks allow an attacker to access sensitive information by **intercepting and altering communications** between an end-user and web servers

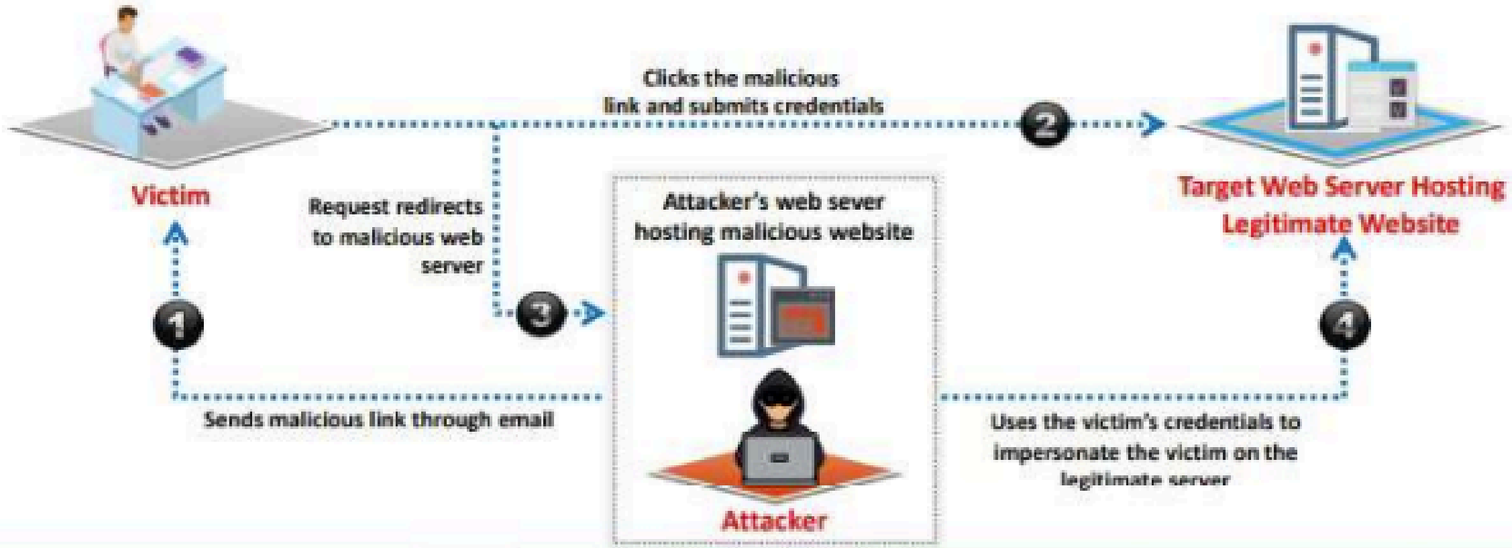
2. An attacker **acts as a proxy** such that all communications between the user and web server passes through him



Web Server Attacks

Phishing Attacks

- The attacker tricks the user to submit **login details** for a website that looks legitimate, and redirects them to the malicious website hosted on the attacker's web server
- The attacker then **steals the credentials** entered and uses them to impersonate the user with the website hosted on the legitimate target server
- Attacker can then perform **unauthorized** or **malicious operations** on the target legitimate website



Web Server Attacks

Website Defacement

- Web defacement occurs when an intruder **maliciously alters the visual appearance of a web page** by inserting or substituting provocative, and frequently, offending data
- **Defaced pages expose visitors to some propaganda** or misleading information until the unauthorized changes are discovered and corrected
- Attackers use a variety of methods such as **MYSQL injection** to access a site in order to deface it



Web Server Attacks





Web Server Attacks

Website defacement refers to unauthorized changes made to the content of a single web page or an entire website, resulting in changes to the visual appearance of the web page or website. Hackers break into web servers and alter the hosted website by injecting code to add images, popups, or text to a page in such a manner that the visual appearance of the page changes. In some cases, the attacker may replace the entire website instead of just changing a single page.

Defaced pages expose visitors to propaganda or misleading information until the unauthorized changes are discovered and corrected. Attackers use a variety of methods, such as MySQL injection, to access a website to deface it. In addition to changing the visual appearance of the target website, attackers deface websites for infecting the computers of visitors by making the website vulnerable to virus attacks. Thus, website defacement not only embarrasses the target organization by changing the appearance of its website but is also intended to harm its visitors.

Web Server Attacks

Web Server Misconfiguration

- Server misconfiguration refers to **configuration weaknesses in web infrastructure** that can be exploited to launch various attacks on web servers such as directory traversal, server intrusion, and data theft

Web Server Misconfiguration

- Verbose Debug/Error Messages
- Anonymous or Default Users/Passwords
- Sample Configuration and Script Files
- Remote Administration Functions
- Unnecessary Services Enabled
- Misconfigured/Default SSL Certificates

Web Server Misconfiguration Examples

- This configuration allows anyone to view the **server status** page, which contains detailed information about the web server being currently used, including information about the **current hosts** and requests being processed

httpd.conf file on an **Apache** server

```
<Location /server-status>
SetHandler server-status
</Location>
```

- This configuration generates **verbose error messages**

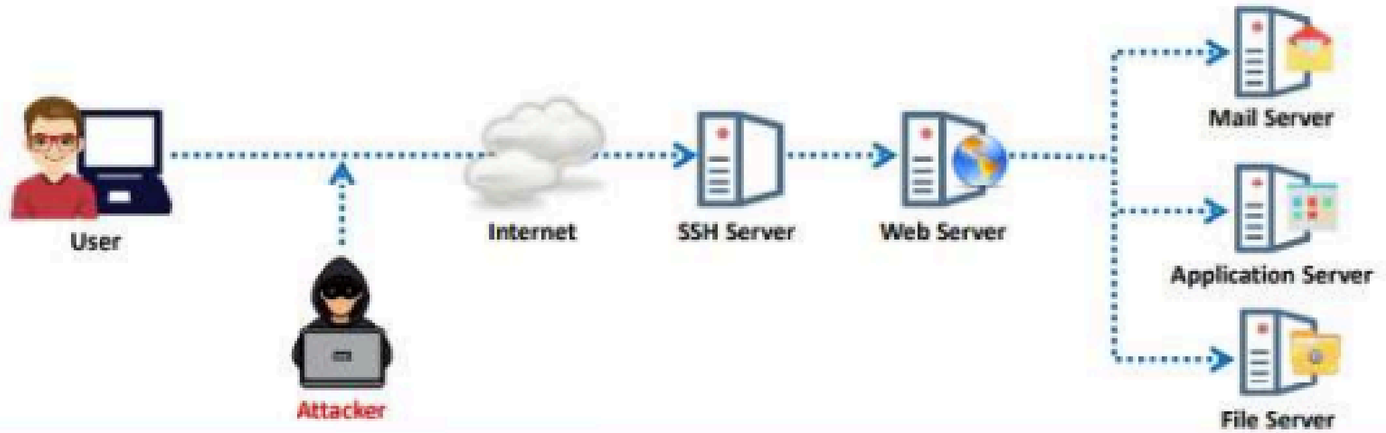
php.ini file

```
display_error = On
log_errors = On
error_log = syslog
ignore_repeated_errors = Off
```

Web Server Attacks

SSH Brute Force Attack

1. SSH protocols are used to create an **encrypted SSH tunnel** between two hosts to transfer unencrypted data over an insecure network
2. Attackers can brute force SSH login credentials to gain **unauthorized access to an SSH tunnel**
3. SSH tunnels can be used to **transmit malwares** and other exploits to victims without being detected



Web Server Attacks

Web Application Attacks

- Vulnerabilities in **web applications** running on a web server provide a broad attack path for compromising the web servers

Parameter/Form Tampering	**Cookie Tampering**	**Unvalidated Input and File Injection Attacks**
Session Hijacking	**SQL Injection Attacks**	**Directory Traversal**
Denial-of-Service (DoS) Attack	**Cross Site Scripting (XSS) Attacks**	**Buffer Overflow Attacks**
Cross Site Request Forgery (CSRF) Attack	**Command Injection Attacks**	**Source Code Disclosure**

Note: For complete coverage of web application attacks refer to Module 14: Hacking Web Applications

Web Server Attacks

Even if web servers are configured securely or are secured using network security measures such as firewalls, a poorly coded web application deployed on the web server may provide a path for an attacker to compromise the web server's security. If web developers do not adopt secure coding practices while developing web applications, attackers may be able to exploit vulnerabilities and compromise web applications and web server security. An attacker can perform different types of attacks on vulnerable web applications to breach web server security.

SQL Injection Attacks: SQL injection exploits the security vulnerability of a database for attacks. The attacker injects malicious code into the strings, which are later passed on to the SQL server for execution.

Web Server Attacks

Directory Traversal: Directory traversal is the exploitation of HTTP through which attackers can access restricted directories and execute commands outside of the web server's root directory by manipulating a URL.

Command Injection Attacks: In this type of attack, a hacker alters the content of the web page by using HTML code and by identifying the form fields that lack valid constraints.

Denial-of-Service (DoS) Attack: A DoS attack is intended to terminate the operations of a website or server to make it unavailable for access by its intended users.

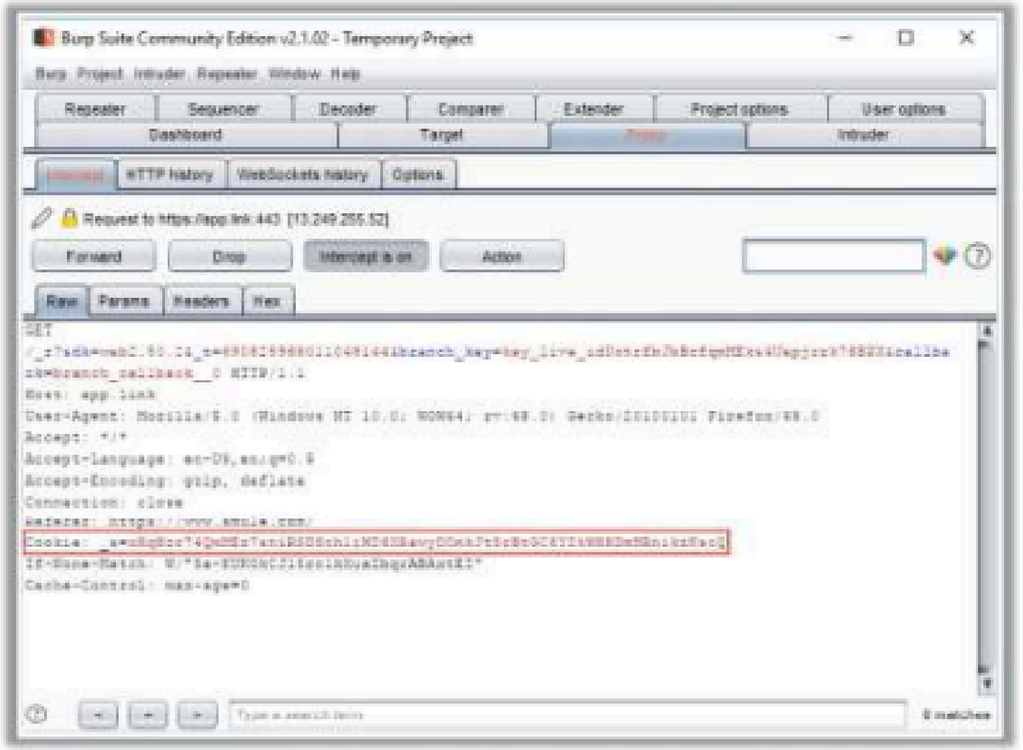
Cross-Site Scripting (XSS) Attacks: In this method, an attacker injects HTML tags or scripts into a target website.

Web Server Attacks

Session Hijacking

- Sniff valid session IDs to **gain unauthorized access** to the web server to snoop data
- Use session hijacking techniques such as session fixation, session sidejacking, Cross-site scripting, etc., to **capture valid session cookies and IDs**
- Use tools such as **Burp Suite**, **JHijack**, **Ettercap**, etc. to automate session hijacking

Note: For complete coverage of Session Hijacking concepts and techniques refer to Module 11: Session Hijacking



What is Patch Management?

What is Patch Management?

- "Patch management is a process used to fix known vulnerabilities by ensuring that the **appropriate patches** are installed on a system"

An automated patch management process

Detect	Use tools to detect missing security patches
Assess	Asses the issue(s) and associated severities by mitigating the factors that may influence the decision
Acquire	Download the patch for testing
Test	Install the patch first on a testing machine to verify the consequences of the update
Deploy	Deploy the patch to the computers and ensure that the applications are not affected
Maintain	Subscribe to get notifications about vulnerabilities as they get detected

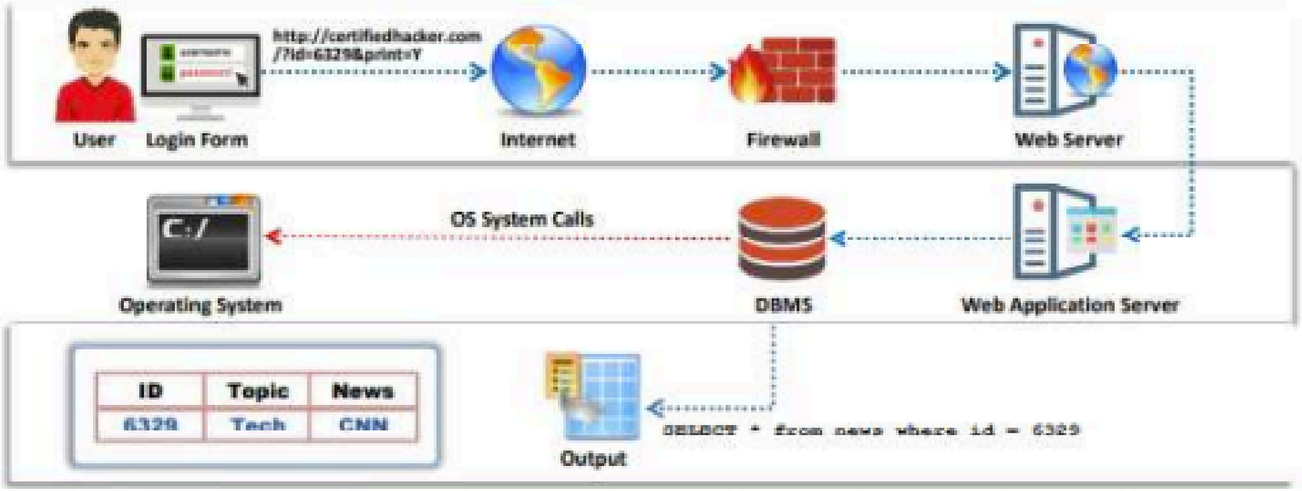
Presentation by Harshit Bhardwaj

Hacking Web Applications

Introduction to Web Applications

- Web applications provide an **interface between end users and web servers** through a set of web pages that are generated at the server end or contain script code to be executed dynamically within the client web browser
- Though web applications enforce certain **security policies**, they are vulnerable to various attacks such as SQL injection, cross-site scripting, and session hijacking

How Web Applications Work



Thank you

PRESENTATION BY HARSHIT BHARDWAJ